IMPERIAL

NLP Coursework

Detecting Patronising and Condescending Language (PCL)

Author
Amin Abu Alhassan
CID: 06045614

GitHub Repository
https://github.com/amin-abualhassan/nlp-pcl
Leaderboard name: `Filasteen_Horra`

NLP (70016)
Imperial College London
2026

Introduction

Code repository: `https://github.com/amin-abualhassan/nlp-pcl`

This report documents my solution for SemEval "Don't Patronize Me!" NLP coursework: detecting patronising or condescending language (PCL) in English news paragraphs [1]. The task is evaluated using the **F1 score of the positive class** (PCL), so the pipeline is designed around handling class imbalance and choosing a decision threshold that is tuned for that metric.

The overall approach is a DeBERTa-v3-large classifier trained with 5-fold stratified cross-validation, with an optional auxiliary head that learns the 7 fine-grained PCL categories when those labels are available. I then ensemble the fold models at inference time and select the final decision threshold using out-of-fold predictions.

Exercise 1 : Critical Paper Review

Q1) What are the primary contributions of this work?

1. **A new dataset.** The main contribution is the *Don't Patronize Me!* dataset: **10,637** English news paragraphs about potentially vulnerable communities, sampled in a structured way using **10 vulnerability-related keywords** across **20 countries** [2].
2. **A clear labeling scheme for a fuzzy thing.** They make "patronising / condescending language" usable by defining a **two-level taxonomy**: **7 fine-grained categories** grouped into **3 higher-level types** (Saviour, Expert, Poet), with definitions and examples [2].
3. **A two-step annotation setup.** First, they do **paragraph-level PCL labeling** (with a "borderline" option), and combine annotator decisions into a **5-point scale** so uncertainty/disagreement isn't thrown away. Then they do **span-level annotation** (marking the exact PCL spans) and assign category labels to those spans using BRAT [2].
4. **Baselines and useful diagnostics.** They provide baseline experiments for two tasks: **binary PCL detection (Task 1)** and **multi-label category prediction (Task 2)** [2]. They also go beyond scores by discussing which categories are harder and showing example errors, which helps future work see where models fail.

Q2) What technical strengths justify publication?

- **They operationalise something subjective.** PCL is messy and context-dependent, but the paper makes it concrete: it explains what they count as PCL *and* what they explicitly don't (e.g., describing suffering $\neq$ automatically condescending; openly hateful/offensive language is out of scope).
- **The annotation design is thoughtful.** They accept that ambiguity is inevitable, so they include a **borderline** label, preserve uncertainty via a **5-point scale**, and use a referee for hard disagreements. That's a sensible setup for a subjective phenomenon.
- **They're transparent about reliability.** They report agreement numbers for both paragraph-level labels and category labels. For dataset papers, that kind of transparency is important because it tells you how noisy the supervision really is.
- **Baselines are reasonable and evaluation matches the task.** They compare classical models, a BiLSTM, and pretrained LMs (including RoBERTa). For Task 1 they report **precision/recall/F1 on the positive class**, which is the right focus for an imbalanced detection problem. They also include concrete misclassification examples, which is helpful for understanding failure modes [2].

Q3) What are the key weaknesses / missing evidence?

- **Transformer baseline reproducibility isn't complete.** They give some training details (epochs, batch size, seed), but not the full recipe needed to reproduce results exactly (learning rate and schedule, max sequence length/truncation choices, preprocessing, etc.). A minimal fix is publishing full configs plus runnable scripts.
- **No generalisation/robustness checks despite structured sampling.** Since the dataset spans keywords, countries, and time, it would be very informative to test **leave-one-keyword-out** or **leave-one-country-out**. As written, it's mostly "in-dataset CV," so we don't learn how stable models are across those axes, especially on out-of-distribution data.
- **They show human disagreement, but don't unpack it.** The paper reports only **moderate agreement** for paragraph-level labeling (and it improves when borderline cases are removed), but it doesn't really analyse what drives disagreement or provide richer borderline guidance/examples.
- **Span supervision exists but isn't exploited in baselines.** They collect span-level annotations, but Task 2 baselines treat categorisation as **paragraph-level multi-label classification** and ignore spans. That's fine as a baseline choice, but it leaves an evidence gap: we don't learn how much span modelling helps (even a simple sequence tagging baseline would answer this).

Exercise 2 : Exploratory Data Analysis (EDA)

I used two different EDA techniques: (1) a structure + data quality (length, truncation, artifacts, leakage checks), and (2) a label + metadata audit (class imbalance, label mapping, keyword shift across splits, and the optional 7-category labels). Both directly shaped my final training/inference pipeline.

Technique A: Structure & data-quality audit (length, truncation, artifacts, leakage)

Visual and Tabular Evidence

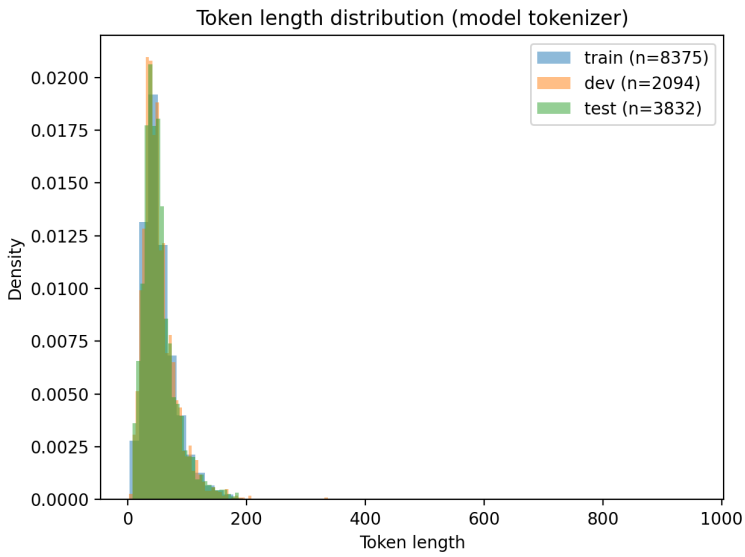

Figure 1: Token length distribution (train/dev/test) using the same DeBERTa tokenizer used in the final pipeline [3], [4], [5].

Table 1: Token length percentiles (DeBERTa tokenizer).

Split	N	P50	P75	P90	P95	P99	Max
train	8375	55	101	149	180	201	956
dev	2094	50	93	139	172	196	337

Table 2: Estimated truncation rates for common `max_length` choices (DeBERTa tokenizer).

MaxLen	Train %	Dev %	Test %
128	2.69	2.20	2.95
192	0.20	0.19	0.16
256	0.07	0.05	0.08

Analysis

- The dataset is mostly short-to-medium paragraphs: the median is **55 tokens** in train and **50 tokens** in dev (Table 1).
- Even the long tail is manageable: the 99th percentile is 201 tokens. Dev is similar (Table 1).
- Truncation is negligible at `max_length=192`: about **0.20%** of train, **0.19%** of dev, and **0.16%** of test would be truncated (Table 2).
- Quick data-quality checks also looked fine:
  - **Duplicates/leakage check:** the script found **0 exact text duplicates** between train and dev (see `duplicate_summary.*`).
  - **Max length choice:** I used **`max_length=192`** as a practical trade-off: truncation stays extremely low (≈**0.2%** on train; Table 2) while keeping sequence lengths smaller during 5-fold CV + ensembling.

Impact Statement

- **Max length choice:** I used **`max_length=192`** as a compute/coverage trade-off: it keeps truncation extremely low (≈**0.2%** on train) while being cheaper than 256/384/512. This matters because the project uses 5-fold CV and an ensemble.
- **Preprocessing:** I kept preprocessing minimal and reproducible. The only "must-have" step is defensive sanitisation (`NaN` → empty string) to avoid tokenizer crashes.
- **Leakage risk:** since train–dev duplicates are zero, I didn't need complicated de-duplication logic or special split handling. That kept the pipeline cleaner and easier to reproduce.

Technique B: Label & metadata audit (imbalance, label scale, keyword effects, aux categories)

Visual/Tabular Evidence

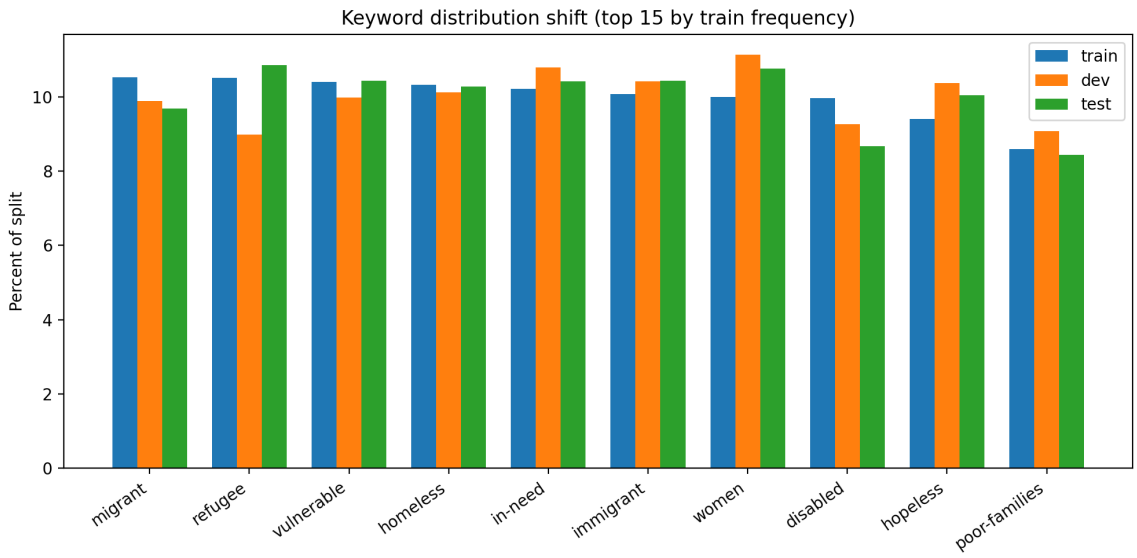

Figure 2: Keywords with the biggest train→dev shift in PCL rate. This is a warning sign for topic/keyword shortcuts and split sensitivity.

Table 3: Binary class balance after mapping label scale to y: labels 0–1 → 0, labels 2–4 → 1.

Split	N	Pos (y=1)	Pos %
train	8375	794	9.48
dev	2094	199	9.50

Table 4: Aux 7-category **multi-label** distribution among **train positives that have aux labels** (prevalence is *within this subset*, not over all train).

Category	Count	Prevalence %
Unbalanced_power_relations	378	58.8
Compassion	219	34.1
Authority_voice	65	10.1
Shallow_solution	65	10.1
Presupposition	63	9.8
Metaphors	55	8.6
The_poorer_the_merrier	14	2.2

Analysis

- **Strong class imbalance:** only about **9.5%** of examples are PCL in both train and dev (Table 3). So accuracy is basically useless here; the metric has to focus on the positive class.
- **Label scale matters:** the original labels are 0–4, and the binary target comes from a thresholded mapping (0–1 → 0, 2–4 → 1). That keeps some ambiguity near the decision boundary. After mapping to binary, you still inherit ambiguity around the decision boundary (so you should not assume threshold 0.5 is optimal).
- **Split sensitivity shows up in the metadata:** for some **keywords**, the observed PCL rate shifts from train to dev (Figure 2). This is a shortcut risk: keyword/topic priors can look predictive even if the underlying PCL cues don't transfer.
- **Aux categories are usable, but not uniform:**
  - Coverage is useful but not complete (many positives have aux labels, but not all), so the pipeline has to handle missing aux rows safely.

- Categories are very imbalanced (Table 4), so if you use them, you need to treat aux supervision carefully (mask missing aux rows, don't let rare categories dominate training instability), and this imbalance is *within* the positive subset with aux labels.

Impact Statement

- **Metric + thresholding:** because positives are rare and borderline exists, I optimised and reported **F1 on the positive class**, and I used **out-of-fold threshold tuning** (rather than fixing 0.5). This directly matches how the shared task is judged.
- **Cross-validation design:** I used **5-fold stratified CV** so each fold sees a similar positive rate (important when positives are ∼9.5%).
- **Ensembling:** keyword effects mean any single split can be a bit misleading. A fold ensemble is a cheap way to reduce variance and "overfit to one slice of the dataset".
- **Optional aux head:** the 7-category labels look like a sensible "extra learning signal" (different flavours of PCL), so I added an **optional auxiliary multi-label head** that can be trained only when aux labels are available (masking missing aux rows). This is a controlled way to use the extra annotation without breaking the main binary objective.

Exercise 3 : Proposed approach

Goal. The organiser baseline is RoBERTa-base, evaluated using **F1 on the positive (PCL) class** [1], [6]. My submitted approach keeps the task definition the same, but upgrades the model and makes training/inference match the metric and the data quirks (rare positives + borderline cases).

High-level idea

I fine-tune a strong transformer encoder (**DeBERTa-v3-large**) for binary PCL detection, then make it more stable by: (i) training with **5-fold stratified cross-validation**, (ii) predicting with a **fold ensemble** (mean of probabilities), and (iii) choosing the decision threshold by **out-of-fold (OOF) threshold tuning** to directly optimise positive-class F1. Optionally, I add a small **auxiliary multi-label head** to predict the 7 PCL categories (only where those labels exist), as a gentle "extra signal" to shape the shared representation without changing the main objective.

```
train/dev text → DeBERTa tokenizer (max_length=192) → 5-fold stratified CV training
        → OOF probabilities → threshold search (step=0.01) for best F1_pos
  → dev/test inference: mean over fold probs → apply tuned threshold → dev.txt /
                                        test.txt
```

Figure 3: Submitted pipeline (training + inference).

Data + targets

Binary label. The original dataset uses a 0–4 label scale with "borderline" in the middle. I map it to binary for Task 4 / Subtask 1 using the standard rule: **0–1 → No PCL** and **2–4 → PCL**. This matches the shared-task framing while acknowledging that the decision boundary is inherently fuzzy (so thresholding matters).

Auxiliary labels (optional). If available, I attach 7-dim multi-label category annotations per paragraph. I prefer the SemEval-style per-paragraph files; if they are missing, I fall back to span-level categories and aggregate spans into a paragraph multi-hot vector (with an optional filter like `min_annotators=2`). To avoid contaminating the auxiliary task, **No-PCL paragraphs are forced to have all-zero aux labels** even if a merge artifact happens.

Model

Backbone: `microsoft/deberta-v3-large` [3], [4].

Pooling: `[CLS]` pooling (used it because it is simple and strong for paragraph classification).

Heads:

- **Binary head:** a linear layer producing one logit (PCL vs No PCL).
- **Aux head (optional):** a linear layer producing 7 logits (multi-label categories).

This is intentionally minimal: I want the gains to come from a better backbone and a training/inference recipe that matches the task, not from adding lots of architectural moving parts.

Training recipe (5-fold CV, imbalance handling, aux supervision)

Cross-validation. I train with **StratifiedKFold (5 folds, shuffle, seed=42)** so every fold sees a similar PCL rate. For each fold, I keep the best epoch using **positive-class F1 on that fold's validation split** (evaluated at threshold 0.5 during training), then save that checkpoint.

Imbalance handling. Because positives are rare, I use **`BCEWithLogitsLoss` with `pos_weight`** (computed per fold as `neg/pos`) for the binary head. This pushes the model to care about missing positives, which is exactly what positive-class F1 penalises.

Auxiliary loss (enabled in BestModel). When auxiliary labels exist, I add a multi-label BCE loss:

$$\mathcal{L} = \mathcal{L}_{bin} + \lambda \, \mathcal{L}_{aux},$$

with a small weight $\lambda = 0.05$. To keep it clean, I apply the aux loss only on rows that actually have aux labels; and (in this submission) I further restrict it to **positive paragraphs only** (`aux_on_positives_only=true`). This avoids wasting capacity on predicting "no category" for negatives and reduces noise from missing/partial aux coverage.

Optimisation / regularisation. I use AdamW with a small learning rate of 2e-5 and standard stabilisers: dropout 0.1, linear warmup, weight decay, gradient clipping, and mixed precision (bf16). The max token length is **192**: based on my EDA, this is a near-lossless compromise (most paragraphs are short) while keeping compute practical for 5-fold training.

Threshold tuning (OOF) and inference

Why threshold tuning is necessary. With (i) strong imbalance and (ii) borderline cases collapsed into a binary label, a fixed threshold of 0.5 is arbitrary. So instead of guessing, I tune it.

How I tune it. After training all folds, I collect **OOF probabilities** for every training example (each example predicted by the model that did not train on it). I then perform a simple grid search over thresholds (step size **0.01**) and pick the threshold that maximises **positive-class F1** on OOF predictions. That single threshold is then used for dev/test outputs.

How I predict dev/test. At inference time I: (1) load each fold checkpoint, (2) compute per-fold probabilities for each example, (3) take the **mean probability across folds**, then (4) apply the tuned threshold to write `dev.txt` and `test.txt` in the required 0/1 line format.

Fold weighting (tried, not submitted). I also tried weighting folds by their CV validation F1 (a "trust good folds more" idea, using a soft weighting with $\gamma = 2$), but the final submission uses **no fold weighting** (plain mean) because it was simpler, the gain was negligible (and maybe not even statistically significant) and did not justify the extra complexity for this submission.

Rationale and expected outcome (vs RoBERTa baseline)

This approach should beat the RoBERTa-base baseline for three straightforward reasons:

1. **Stronger encoder.** DeBERTa-v3-large is a higher-capacity backbone than RoBERTa-base, which usually translates into better sentence/paragraph understanding when fine-tuned carefully.
2. **Training matches the metric.** Positive-class F1 is sensitive to the precision/recall trade-off. Using `pos_weight` and **OOF threshold tuning** directly targets that trade-off instead of relying on a default 0.5 cutoff.
3. **Stability under split quirks.** CV + fold ensembling reduces variance and makes the system less sensitive to which slice of the dataset you happened to train on (useful when keyword/topic effects shift between splits, as seen in my EDA).

Overall, the novelty here is not a fancy architecture. It's a *tight, focused, and well thought out approach that is well suited for the task at hand*: better backbone + CV ensemble + metric-aligned thresholding, with a small auxiliary head to use the extra supervision without derailing the main task.

Exercise 4 : Implementing and training the proposed approach

Leaderboard name: `Filasteen_Horra`
Repository: `https://github.com/amin-abualhassan/nlp-pcl`
Models backup (if you face lfs issues): `https://drive.google.com/drive/folders/1FAUTcZRYLRKM-L8iZt_-mGZiH9xzGy89?usp=drive_link`

I implemented the full training and inference pipeline in the repository above. The code follows the approach described in Exercise 3 (DeBERTa-v3-large, 5-fold stratified CV, optional auxiliary 7-category supervision, and out-of-fold threshold tuning for positive-class F1).

BestModel folder. The repository contains a `BestModel/` folder which includes: (i) the exact configuration used (`BestModel/config.yaml`), (ii) the shipped trained model artifacts as a split archive (`BestModel/model_archive/`), and (iii) a short set of commands to restore the fold checkpoints and reproduce the final outputs.

Training entry-point. Training is implemented as a script-first workflow. The command used to train the 5-fold CV models is:

```
python3 scripts/train_cv.py --config configs/default.yaml
```

This writes fold checkpoints and logs under: `models/outputs/<run_name_timestamp>/` (one folder per run), including each fold's `model.pt` and tokenizer.

Restoring the trained model checkpoints. Because the fold checkpoints are large, they are stored in the repo as a split archive (tracked via Git LFS) and restored locally using:

```
bash scripts/restore_models.sh
```

After restoration, the selected run used for submission is available at:
`models/outputs/20260228_165925_deberta_mtl_cv5_lam_sweep_1`.

Reproducing the submission outputs. The final `dev.txt` and `test.txt` are generated via fold-ensemble inference with the selected out-of-fold threshold (not just using one of the folds):

```
python3 scripts/build_dev_test.py --fold_weighting none
```

This rebuilds the submission-format files deterministically from the restored fold checkpoints.

Exercise 5.1 : Global Evaluation

I submitted predictions for the official dev and test sets as plain text files: `dev.txt` and `test.txt` (both are in the repository root).

- **Format:** one prediction per line; each line is either `0` (No PCL) or `1` (PCL).
- **Counts:** `dev.txt` has **2094** lines (matches `dev_df_2.csv`); `test.txt` has **3832** lines (matches `task4_test.tsv`).
- **How they were produced:** fold-ensemble inference from the selected run, then applying the tuned threshold ($t^* = 0.45$) to output hard labels. Rebuilt with:

  ```
  python3 scripts/build_dev_test.py --fold_weighting none
  ```

Exercise 5.2: Local Evaluation (Error Analysis + Additional Evaluation)

I treated the official dev set as my local "test set" (labels available), and used it to understand how the final system behaves beyond a single F1 score. All analyses below are generated by running:

```
python3 scripts/local_eval_stage5.py \
  --selection_report selection_report.json \
  --dev_csv data/dev_df_2.csv \
  --pred dev.txt \
  --pred_alt dev.txt.bak \
  --cats_tsv data/raw/dontpatronizeme_categories.tsv \
  --out_dir reports/local_eval
```

Overall dev behaviour. My final submission uses an out-of-fold tuned threshold $t^* = 0.45$ (selected to maximise F1 on the positive class). On the official dev set, this yields: TP=117, FN=82, FP=73, TN=1822 (Table 5). This corresponds to $p \approx 0.616$, $r \approx 0.588$, and $\text{F1}_{pos} \approx 0.602$.

	Pred 1	**Pred 0**
True 1	117 (TP)	82 (FN)
True 0	73 (FP)	1822 (TN)

Table 5: Confusion matrix on the official dev set at $t^* = 0.45$.

Stability via bootstrap confidence intervals. To check that performance is not overly driven by a small set of examples, I bootstrapped the dev set and recomputed the metrics. The positive-class F1 remains fairly stable, with a 95% bootstrap interval roughly $[0.55, 0.66]$. This supports that the chosen operating point is reasonably robust and reliably beats the baseline.

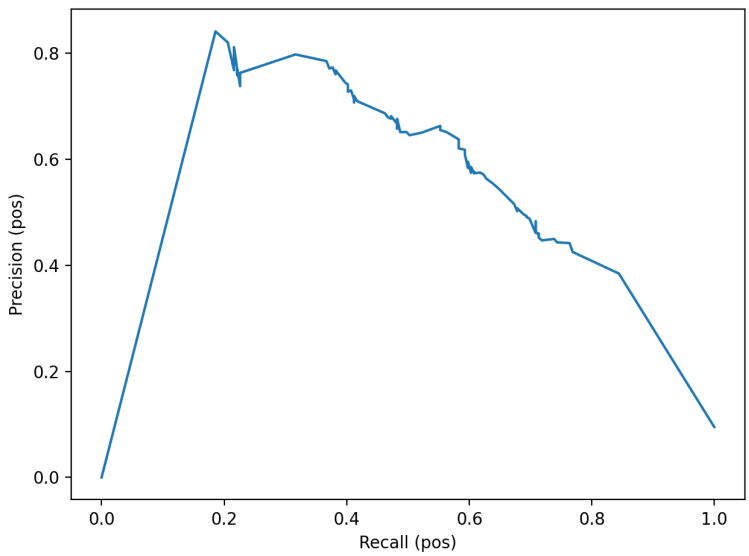

Figure 4: Precision–recall curve on the official dev set using the model's predicted probabilities.

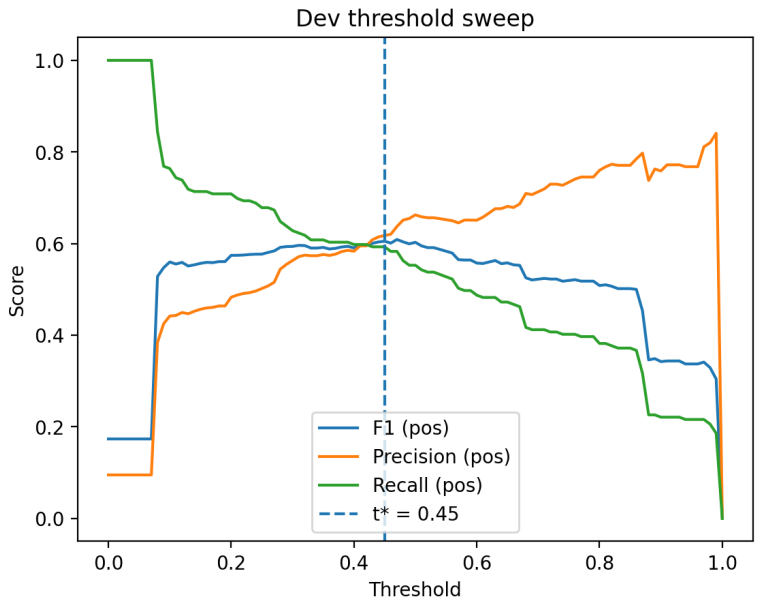

Figure 5: Dev threshold sweep showing the precision/recall/F1 trade-off. The selected $t^* = 0.45$ is close to the dev-optimal region.

Threshold trade-off and PR analysis (additional local evaluation). Because the dataset is highly imbalanced (positives are a minority), I inspected precision–recall behaviour rather than accuracy. Figure 4 shows the PR curve; the average precision (AUPRC) is ≈ 0.63 and ROC-AUC is ≈ 0.93. I also swept thresholds on dev (Figure 5): the best dev F1 occurs near $t \approx 0.47$, which is very close to my chosen $t^* = 0.45$. This is a good sign that the OOF-selected threshold generalises to the dev set without needing dev-specific tuning.

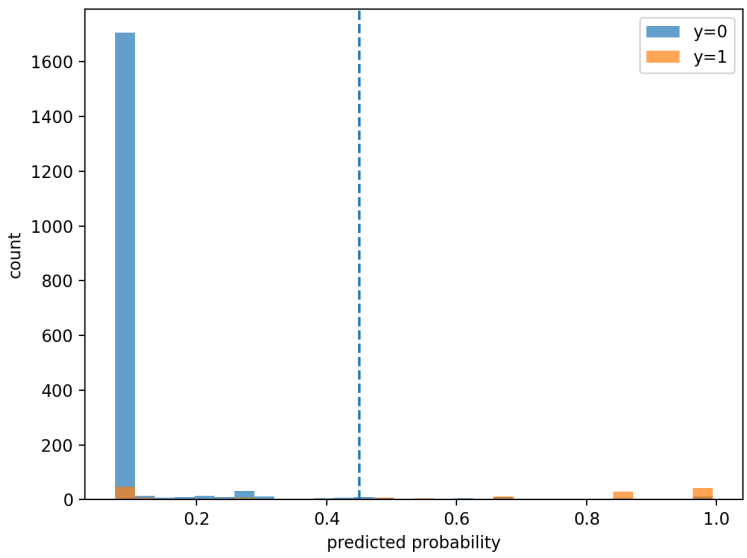

Figure 6: Predicted probability histogram for positives vs negatives on the official dev set.

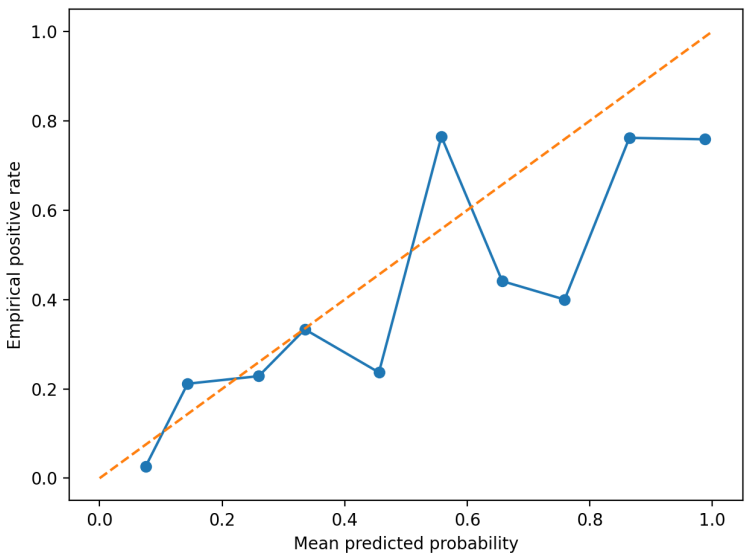

Figure 7: Reliability (calibration) curve for predicted probabilities on the official dev set.

Calibration and score distribution. Figure 6 shows the score histogram split by the true label. Most true negatives concentrate near low scores, while positives are more spread (including many borderline cases). Calibration is not perfect (common for fine-tuned transformers), but the Brier score is ≈ 0.059 and the reliability curve is acceptable for a thresholded classifier (Figure 7). Since the final decision rule is tuned for $\mathrm{F1}_{pos}$, perfect calibration is not required, but these plots help interpret borderline decisions near t^*.

Slice analysis (where the model breaks). I inspected performance on interpretable slices to locate systematic failure modes.

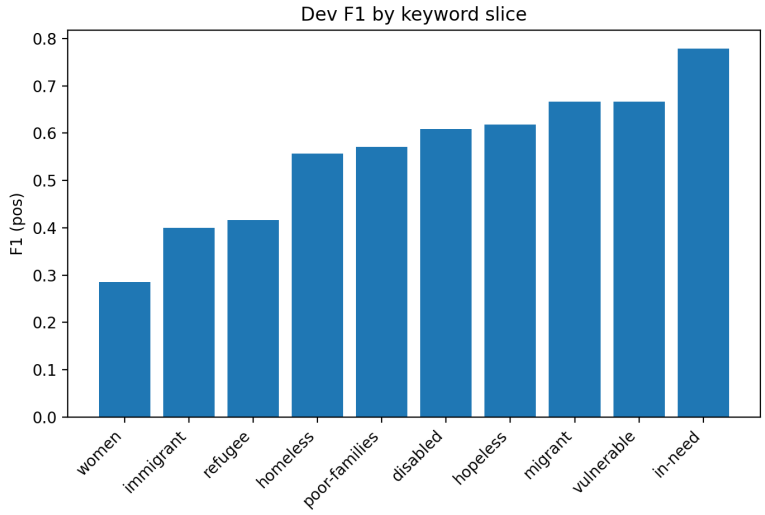

Figure 8: Positive-class F1 across keyword slices on the official dev set (higher is better).

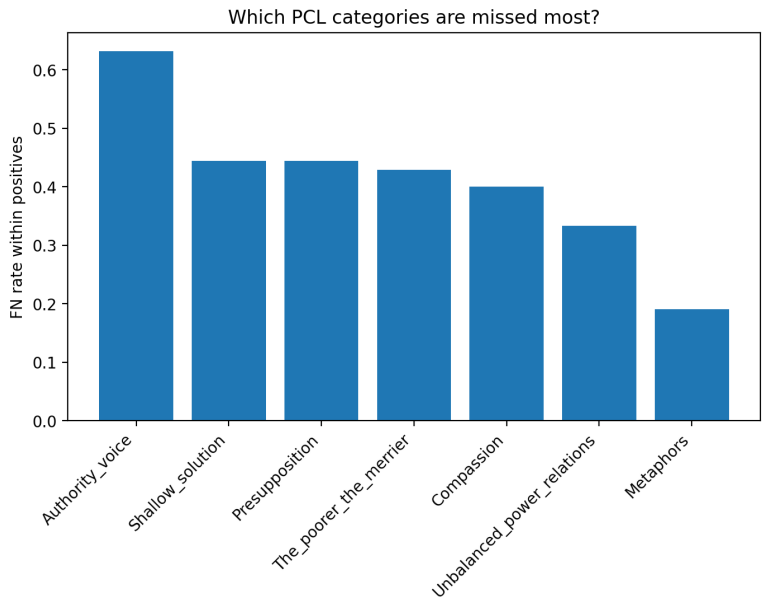

Figure 9: False-negative rate among positives, grouped by span-category tags (using ≥ 2 annotators). Higher means the category is harder for the model.

(1) Label severity (scale 0–4). A key pattern is that recall is much higher for stronger PCL. Within positive label buckets, recall is: label-2 ≈ 0.17 (borderline), label-3 ≈ 0.60, label-4 ≈ 0.76. This suggests the model is reliable on overt condescension, but struggles when PCL is conveyed subtly via framing.

(2) Keyword slices. Some topic slices are consistently harder (Figure 8). The weakest slices include *women*, *immigrant*, and *refugee*, where false negatives are common. This aligns with the error examples: these topics often appear in a neutral "reporting" style where PCL is implied rather than explicit.

(3) Category-level misses using span-category metadata. Using the provided span-level category annotations, I computed which conceptual PCL categories are missed most often. Figure 9 shows that *Authority_voice* has the highest miss-rate, followed by *Unbalanced_power_relations*. This is an important limitation: these categories can be expressed without obvious lexical cues, so the model often assigns very low probability even when the gold label is positive.

Error analysis (manual inspection of failures). I manually inspected the highest-confidence mistakes to understand what language cues the model is relying on.

False positives (FP): "help / charity" narratives. Many top FPs describe donations, shelters, or support initiatives in a non-condescending way. The model appears to sometimes treat generic aid language as a proxy for PCL, which is a spurious cue: it is correlated with PCL in parts of the training data, but not equivalent to condescension.

False negatives (FN): subtle framing without obvious cues. Many top FNs read like neutral reporting (e.g., internships or subsidies aimed at vulnerable groups). These are likely annotated as PCL due to subtle framing (presuppositions, power imbalance, or "speaking for" a group) rather than explicit patronising phrases. This matches the slice results: borderline/implicit PCL (especially Authority_voice) is where recall drops sharply.

Weighted vs unweighted fold ensembling. I also tested a CV-weighted fold ensemble (beta=2.0 weighting), but it only changed 6 predictions out of 2094 on dev and did not materially improve behaviour. Given the tiny difference, I kept the plain mean ensemble for simplicity and reproducibility.

Bibliography

[1] C. Pérez-Almendros, L. Espinosa-Anke, and S. Schockaert, "SemEval-2022 Task 4: Patronizing and condescending language detection," in *Proceedings of the 16th International Workshop on Semantic Evaluation (SemEval-2022)*, Association for Computational Linguistics, 2022, pp. 298–307.

[2] C. Pérez-Almendros, L. Espinosa-Anke, and S. Schockaert, "Don't patronize me! an annotated dataset with patronizing and condescending language towards vulnerable communities," in *Proceedings of the 28th International Conference on Computational Linguistics*, International Committee on Computational Linguistics, 2020, pp. 4891–4902.

[3] P. He, X. Liu, J. Gao, and W. Chen, *Deberta: Decoding-enhanced bert with disentangled attention*, 2020. DOI: `10.48550/arXiv.2006.03654`. arXiv: `2006.03654 [cs.CL]`.

[4] Microsoft, *Microsoft/deberta-v3-large (hugging face model card)*, `https://huggingface.co/microsoft/deberta-v3-large`, 2021. Accessed: Mar. 3, 2026.

[5] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Association for Computational Linguistics, 2020, pp. 38–45.

[6] Y. Liu et al., *Roberta: A robustly optimized bert pretraining approach*, 2019. DOI: `10.48550/arXiv.1907.11692`. arXiv: `1907.11692 [cs.CL]`.